INFO



Selected Anti-Patterns

A wide variety of anti-design patterns have been identified to assist developers in making refactoring decisions. Comprehensive descriptions of each of these patterns can be obtained via links at **https://en.wikipedia.org/wiki/Anti-pattern.**

- **Big ball of mud.** A system with no recognizable structure
- **Stovepipe system.** A barely maintainable assemblage of ill-related components
- **Boat anchor.** Retaining a part of a system that no longer has any use
- **Lava flow.** Retaining undesirable (redundant or low-quality) code because removing it is too expensive or has unpredictable consequences
- **Spaghetti code.** Program whose structure is barely comprehensible, especially because of misuse of code structures
- **Copy and paste programming.** Copying existing code several times rather than creating generic solutions
- **Silver bullet.** Assume that a favorite technical solution will always solve a larger process or problem
- **Programming by permutation.** Trying to approach a solution by successively modifying the code to see if it works

14.6 USER INTERFACE DESIGN PATTERNS

Hundreds of user interface (UI) patterns have been proposed in recent years. Most fall within one of 10 categories of patterns as described by Tidwell [Tid11] and vanWelie [Wel01]. A few representative categories (discussed with a simple example[6]) follow:

Whole UI. Provide design guidance for top-level structure and navigation throughout the entire interface.

Pattern:	**Top-level navigation**
Brief description:	Used when a site or application implements several major functions. Provides a top-level menu, often coupled with a logo or identifying graphic, that enables direct navigation to any of the system's major functions.
Details:	Major functions (generally limited to between four and seven function names) are listed across the top of the display (vertical column formats are also possible) in a horizontal line of text. Each name provides a link to the appropriate function or information source. Often used with the **BreadCrumbs** pattern discussed later.
Navigation elements:	Each function/content name represents a link to the appropriate function or content.

Page layout. Address the general organization of pages (for websites) or distinct screen displays (for interactive applications).

6 An abbreviated pattern template is used here. Full pattern descriptions (along with dozens of other patterns) can be found at [Tid11] and [Wel01].

Pattern:	**Card stack**
Brief description:	Used when several specific subfunctions or content categories related to a feature or function must be selected in random order. Provides the appearance of a stack of tabbed cards, each selectable with a mouse click and each representing specific subfunctions or content categories.
Details:	Tabbed cards are a well-understood metaphor and are easy for the user to manipulate. Each tabbed card (divider) may have a slightly different format. Some may require input and have buttons or other navigation mechanisms; others may be informational. May be combined with other patterns such as **drop-down list, fill-in-the-blanks**, and others.
Navigation elements:	A mouse click on a tab causes the appropriate card to appear. Navigation features within the card may also be present, but in general, these should initiate a function that is related to card data, not cause an actual link to some other display.

E-commerce. Specific to websites, these patterns implement recurring elements of e-commerce applications.

Pattern:	**Shopping cart**
Brief description:	Provides a list of items selected for purchase.
Details:	Lists item, quantity, product code, availability (in stock, out of stock), price, delivery information, shipping costs, and other relevant purchase information. Also provides ability to edit (e.g., remove, change quantity).
Navigation elements:	Contains ability to proceed with shopping or go to checkout.

Each of the preceding example patterns (and all patterns within each category) would also have a complete component-level design, including design classes, attributes, operations, and interfaces. If you have further interest, see [Cho16], [Gas17], [Kei18], [Tid11], [Hoo12], and [Wel01] for further information. The role of anti-patterns and their effects on UX design can be found in [Sch15] and [Gra18].

14.7 MOBILITY DESIGN PATTERNS

Throughout this chapter you've learned that there are different types of patterns and how they can be categorized. By their nature, mobile applications are all about the interface. In many cases, mobile UI patterns [Mob12] are represented as a collection of "best of breed" screen images for apps in a variety of different categories. Typical examples might include:

Check-in screens. How do I check in from a specific location, make a comment, and share comments with friends and followers on a social network?

Maps. How do I display a map within the context of an app that addresses some other subject? For example, review a restaurant and represent its location within a city.

Popovers. How do I represent a message or information (from the app or another user) that arises in real time or as the consequence of a user action?

Sign-up flows. How do I provide a simple way to sign in or register for information or functionality?

Custom tab navigation. How do I represent a variety of different content objects in a manner that enables the user to select the one she wants?

Invitations. How do I inform the user that he must participate in some action or dialog? Typical examples might include making use of a dialog box, a tool tip, or a self-playing video demo.

Additional information about mobile UI patterns can be found in [Nei14], [Hoo12], [McT16], [Abr17], and [Pun17]. In addition to UI patterns, Meier and his colleagues [Mei12] propose a variety of more general pattern descriptions for mobile apps.

14.8 SUMMARY

Design patterns provide a codified mechanism for describing problems and their solution in a way that allows the software engineering community to capture design knowledge for reuse. A pattern describes a problem, indicates the context enabling the user to understand the environment in which the problem resides, and lists a system of forces that indicate how the problem can be interpreted within its context and how the solution can be applied. In software engineering work, we identify and document generative patterns. These patterns describe an important and repeatable aspect of a system and then provide us with a way to build that aspect within a system of forces that is unique to a given context.

Architectural patterns describe broad-based design problems that are solved using a structural approach. Data patterns describe recurring data-oriented problems and the data modeling solutions that can be used to solve them. Component patterns (also referred to as design patterns) address problems associated with the development of subsystems and components, the ways in which they communicate with one another, and their placement within a larger architecture. Software anti-patterns describe commonly occurring solutions to a problem that lead to the creation of software products with poor quality. Interface design patterns describe common user interface problems and their solution with a system of forces that includes the specific characteristics of end users. Mobile patterns address the unique nature of the mobile interface and functionality and control elements that are specific to mobile platforms.

A framework provides an infrastructure in which patterns may reside and idioms describe programming language–specific implementation detail for all or part of a specific algorithm or data structure. A standard form or template is used for pattern descriptions. A pattern language encompasses a collection of patterns, each described using a standardized template and interrelated to show how these patterns collaborate to solve problems across an application domain.

Pattern-based design is used in conjunction with architectural, component-level, and user interface design methods. The design approach begins with an examination of the requirements model to isolate problems, define context, and describe the system of forces. Next, pattern languages for the problem domain are searched to determine if patterns exist for the problems that have been isolated. Once appropriate patterns have been found, they are used as a design guide.

PROBLEMS AND POINTS TO PONDER

14.1. Discuss the three "parts" of a design pattern, and provide a concrete example of each from some field other than software.

14.2. What is the difference between a pattern and an anti-pattern?

14.3. How do architectural patterns differ from component patterns?

14.4. What is a framework, and how does it differ from a pattern? What is an idiom, and how does it differ from a pattern?

14.5. Using the design pattern template presented in Section 14.1.3, develop a complete pattern description for a pattern suggested by your instructor.

14.6. Develop a skeletal pattern language for a sport with which you are familiar. You can begin by addressing the context, the system of forces, and the broad problems that a coach and team must solve. You need only specify pattern names and provide a one-sentence description for each pattern.

14.7. When Christopher Alexander says, "good design cannot be achieved simply by adding together performing parts," what do you think he means?

14.8. Using the pattern-based design tasks noted in Section 14.2.3, develop a skeletal design for the "interior design system" described in Section 12.3.2.

14.9. Build a pattern-organizing table for the patterns you used in Problem 14.8.

14.10. Using the design pattern template presented in Section 14.1.3, develop a complete pattern description for the **Kitchen** pattern mentioned in Section 14.3.

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

Three

QUALITY AND SECURITY

In this part of *Software Engineering: A Practitioner's Approach,* you'll learn about the principles, concepts, and techniques that are applied to manage and control software quality. These questions are addressed in the chapters that follow:

- What are the generic characteristics of high-quality software?
- How do we review quality, and how are effective reviews conducted?
- What is software quality assurance?
- What strategies are applicable for software testing?
- What methods are used to design effective test cases?
- Are there realistic methods that will ensure that software is correct?
- How can we manage and control changes that always occur as software is built?
- What measures and metrics can be used to assess the quality of requirements and design models, source code, and test cases?
- How do we ensure the security concerns for a product are being addressed during the software life cycle?

Once these questions are answered, you'll be better prepared to ensure that high-quality software has been produced.